

Performance Report

Pre optimization:

JIT Inline (Multi-Access)

Prior to any optimization utilizing Single Access JIT, data gathered by a script that counts how long it takes to generate 10,000,000 dummy moves resulted in JIT Inline (Multi-Access) taking approximately 0.00271s. Multi Accessing JIT utilizes indexing multiple times when needed, which uses system resources.

Post optimization:

JIT Inline (Single-Access)

After optimizing using Multi Access JIT, once again data was gathered utilizing a script to generate 10,000,000 dummy moves. This resulted in JIT Inline (Single-Access) taking approximately 0.00268s. These statistics display a 1.12% increase in speed, which is significantly greater in the context of generating hundreds of thousands of moves in an instant. For example, if one single move were to take 0.001 seconds to generate, 800,000 moves would take 13.33 minutes to generate. Similarly, if we were to use this 1.12% increase in speed that JIT Inline (Single-Access) yields, this would result in the same demand taking 13.18 minutes, shedding an extra nine seconds from the processing time. Single Accessing JIT consists of indexing once and storing it, minimizing the need for multiple indexing, which in turn saves system resources. A snippet of this stress test program can be seen below.

```
-----
LEADERBOARD (Fastest to Slowest):
-----
#1. V7: JIT Inline (Single-Access)      | 0.00242s | (1.0x slower)
#2. V6: JIT Inline (Multi-Access)      | 0.00254s | (1.1x slower)
#3. V3: JIT Loop + JIT Func            | 0.00297s | (1.2x slower)
#4. V5: Pure Python Inline (Single-Access) | 3.73951s | (1544.6x slower)
#5. V2: Pure Python Loop + JIT Func    | 3.91100s | (1615.4x slower)
#6. V1: Pure Python Loop + Pure Python Func | 4.94418s | (2042.1x slower)
#7. V4: Pure Python Inline (Multi-Access) | 5.31858s | (2196.8x slower)
```

